

Development of a Multimodal System for Converting PDF Documents into Structured Markdown

Noor Martini

May 7, 2026

Contents

1	Introduction	3
1.1	Problem Statement	4
1.2	Motivation	6
1.3	Research Objectives and Questions	9
1.4	Scope and Delimitations	10
1.5	Thesis Contributions	11
1.6	Thesis Structure	12
2	Background and Fundamentals	12
2.1	PDF Format and Parsing Challenges	12
2.2	Markdown Format	14
2.3	Large Language Models	14
2.4	Multimodal Processing Strategies	14
3	Related Work	14
3.1	Traditional PDF Processing Tools	14
3.2	Modern Document Conversion Tools	14
3.3	LLM-based Document Understanding	15
3.4	Evaluation Methods for Document Conversion	15
4	Methodology	15
4.1	Research Design	15
4.2	Dataset and Ground Truth Preparation	15

4.3	Evaluation Metrics	15
4.4	Experimental Design	15
5	System Design and Implementation	16
5.1	Architecture Overview	16
5.2	PDF Processing Module	16
5.3	LLM Integration	16
5.4	Post-processing Pipeline	16
5.5	Implementation Details	17
6	Results and Evaluation	17
6.1	Experimental Results Overview	17
6.2	Quantitative Analysis	17
6.3	Qualitative Analysis	17
6.4	Strategy Comparison Summary	17
6.5	Limitations and Observations	18
7	Discussion	18
7.1	Interpretation of Results	18
7.2	Threats to Validity	18
7.3	Practical Implications	18
7.4	Comparison with Related Work	18
8	Conclusion	18
8.1	Summary of Findings	18
8.2	Contributions and Impact	18
8.3	Limitations of the Study	18
8.4	Future Work	18
A	Implementation Code Snippets	20
B	Complete Results Tables	20
C	Sample Documents and Outputs	20
D	Prompt Templates	20

1 Introduction

The rapid expansion of digital information has fundamentally changed how organizations store, share, and process documents. The Portable Document Format (PDF) has become the de facto standard for document exchange across industries including law, finance, healthcare, academia, and government. PDF remains the dominant archival and distribution format across industries, and its continued dominance makes automated processing an increasingly critical challenge. Despite this ubiquity, the PDF format presents a fundamental paradox: it excels at preserving visual fidelity for human readers while simultaneously obstructing automated, machine-readable access to the information it contains [4].

Existing approaches to PDF-to-Markdown conversion fall broadly into three categories. Traditional text-based parsers such as PyMuPDF and pdfplumber [10] extract raw text from native PDFs by reading embedded glyph data directly, but discard layout structure in the process. Optical Character Recognition (OCR)-based tools such as Tesseract [9] address scanned documents by treating pages as flat images and recognizing characters pixel by pixel, recovering characters without semantic hierarchy. A newer generation of AI-powered systems, including Marker, Microsoft MarkItDown, LlamaParse, and multimodal OCR models such as Mistral OCR [12], applies deep learning to recover more structure, handling tables, formulas, and multi-column layouts with greater fidelity. A common pipeline combines these stages: scanned pages are first processed with an OCR engine, then passed to a layout-aware conversion tool, and finally cleaned with regex-based post-processing.

Despite these advances, a fundamental limitation persists across all current tools: each applies a single, fixed extraction strategy to the entire document. A tool optimized for text-heavy academic papers will apply the same logic to a page containing only a scanned diagram or a dense mathematical formula. This one-size-fits-all assumption breaks down on real-world documents, which routinely mix plain text, tables, images, source code, and equations within the same file — and sometimes within the same page. No existing open-source tool selects its extraction strategy adaptively based on what each page actually contains.

This thesis addresses this paradox by developing and evaluating a multimodal, content-aware system that converts PDF documents into structured Markdown. Unlike existing tools that apply a single fixed extraction strategy to an entire document, the proposed system analyzes each page individually and selects the most appropriate strategy based on the detected content type: plain text, images, tables, hyperlinks, page numbers, source code, or mathematical formulas. This per-page adaptive approach is powered by LM Studio, a local LLM runtime, ensuring full reproducibility without dependence on external cloud

services. The conversion from PDF to Markdown is not merely a format transformation; it is a semantic reconstruction: recovering headings, lists, tables, links, code blocks, and document hierarchy from a format that encodes position and appearance, but not meaning. This chapter presents the problem motivating this research, the objectives driving the design of the proposed system, and the scope within which the work is situated.

1.1 Problem Statement

Difficulty of Extracting Structured Content from PDFs

The PDF specification, originally developed by Adobe Systems in the early 1990s [4], was designed as a faithful electronic representation of printed pages. Its internal structure stores content as a flat sequence of graphical operations: glyphs rendered at specific coordinates with particular fonts and sizes [4], functioning more like instructions on how to visually produce a page than a structured representation of content [2]. Unlike HTML or XML, PDF has no native concept of a paragraph, heading, list item, or table cell [4]. As Subramani et al. [5] observe, automated document understanding must bridge the gap between documents *made for humans* and systems that can parse machine-readable structure, a challenge that requires combining both computer vision (CV) and natural language processing (NLP) architectures into a unified solution.

Extracting the logical reading order alone is non-trivial: multi-column layouts, footnotes, side-bars, and mixed-language documents all violate the assumption that positional order corresponds to reading order [3]. The problem compounds when documents contain embedded images, complex tables spanning multiple pages, mathematical formulas encoded as bitmaps, or watermarks overlaid on text [3]. These structural complexities mean that a simple text dump from a PDF extractor produces output that is fragmented, mis-ordered, or semantically meaningless without further processing [11].

Limited OCR Accuracy for Complex Layouts

Optical Character Recognition (OCR) is the classical solution for image-based PDFs — scanned documents where text exists only as pixels. However, OCR accuracy degrades sharply when documents contain complex layouts. Subramani et al. [5] identify document layout analysis as a prerequisite step: a model must first identify and categorize content regions (text blocks, figures, tables, equations) before OCR can be meaningfully applied. Traditional OCR pipelines, including widely-used tools such as Tesseract [9], treat the document page as a flat image and struggle to infer the semantic role of detected text regions.

This limitation is quantitatively demonstrated by Zhu et al. [1], who show that applying OCR directly to complex documents without prior layout segmentation yields a normalized edit distance of 0.63, whereas adding a layout segmentation precursor reduces it to 0.38 (lower is better) — a substantial improvement achieved solely by first decomposing the page into semantically coherent regions.

A primary driver of this degradation is multi-column layout: articles spanning multiple columns cause OCR engines to merge or mis-order text across columns, violating reading order, and Zhu et al. [1] demonstrate that preserving human read-order in such documents requires a dedicated segmentation stage before any character recognition is attempted. Modern surveys and comparative studies confirm that even state-of-the-art OCR models face challenges with dense multi-column text, distorted or rotated text, tabular content requiring row-column alignment, and mathematical notation [5, 10]. In a study of end-to-end PDF-to-Markdown conversion, Blecher et al. [6] note that the PDF format causes “a loss of semantic information, particularly for mathematical expressions,” a point reinforced by Darvishy [2], who observes that no existing tool provides mechanisms to interpret or describe mathematical formulas and scientific graphs. This underscores that the limitation is not purely a pixel-level recognition problem but an inherently structural one — and directly motivates the content-aware, per-page strategy developed in this thesis.

Need for Conversion to Markdown for Accessibility and Version Control

The need to convert documents to Markdown arises from a convergence of practical demands across software development, scientific publishing, and artificial intelligence. Markdown is a lightweight markup language created by John Gruber in 2004, designed to be readable in raw form while converting cleanly to HTML and other structured formats [15]. Cassel et al. [7] characterize it as ideal for machine-readability due to its simple syntax and structured format, noting that community extensions further support tables and advanced mathematical notation, making it a well-suited target for document conversion pipelines.

Three properties make Markdown especially valuable as a target representation:

1. **Version control compatibility.** Because Markdown is plain text, every change is tracked, attributed, and auditable in version control systems such as Git, treating a document as a fully versioned project [14].
2. **Machine-readability and structured format.** Markdown’s simple syntax and structured format preserve context and enhance comprehensibility for both humans and automated systems, with community extensions further supporting tables and

advanced mathematical notation [7].

3. **Portability and long-term accessibility.** Markdown files can be opened with virtually any text editor on any operating system, with no dependency on proprietary software. Even if a specific application stops working, the content remains fully readable — an important consideration for documents that need to be preserved indefinitely [15].
4. **LLM and RAG pipeline integration.** The growth of large language models (LLMs) and retrieval-augmented generation (RAG) architectures has created strong demand for structured plain-text document representations. Structured Markdown preserves semantic hierarchy — headings, lists, tables — that improves retrieval precision and context quality when documents are ingested into AI pipelines [13].

1.2 Motivation

Growing Importance of Document Digitization

The global drive toward document digitization is accelerating across all sectors. Increasing digital transformation investments and the proliferation of AI-driven workflows are creating strong demand for structured and searchable document storage. Organizations are no longer merely scanning documents but are increasingly requiring semantic understanding: the ability to extract, query, and transform document content programmatically. PDF-to-Markdown conversion sits at the intersection of this shift, serving as the bridge between static archival documents and the dynamic, queryable knowledge bases that modern workflows demand.

Advantages of Markdown as a Structured Representation

Markdown offers a unique balance between human readability and machine processability. Unlike proprietary formats such as Word or PDF, Markdown does not rely on closed specifications and can be used to create websites, documents, books, presentations, and technical documentation, and converted between these formats using tools such as Pandoc. Its widespread adoption across platforms such as GitHub and Reddit means Markdown files can be opened with virtually any application on any operating system, and remain readable even if the software used to create them is no longer available. This flexibility allows document content to be *written once and adapted for various platforms without reformatting* [15].

Table 1 summarizes the key properties of PDF, Word, and Markdown as document formats, highlighting why Markdown is the preferred target representation for automated document

processing pipelines.

Table 1: Comparison of PDF, Word (.docx), and Markdown as document formats

Property	PDF	Word (.docx)	Markdown
Human readable (raw)	No	No	Yes
Machine parseable	Difficult	Moderate	Easy
Version control (Git)	Poor	Poor	Excellent
Format portability	High	Low	High
Preserves visual fidelity	Excellent	Good	Limited
LLM/RAG compatibility	Poor	Moderate	Excellent
Requires proprietary software	Partial	Yes	No
Long-term archival stability	High	Moderate	High

Potential of LLMs in Understanding Context and Structure

The emergence of large language models with vision capabilities has opened new possibilities for document understanding that transcend traditional OCR-based approaches. Blecher et al. [6] introduced Nougat (Neural Optical Understanding for Academic Documents), a Visual Transformer model that converts images of PDF pages directly into a structured markup language. Their key insight is that an end-to-end model trained on paired PDF-markup data can infer semantic structure (headings, equations, tables, and citations) without requiring an explicit layout analysis step. This demonstrated the viability of replacing multi-stage OCR pipelines with a single multimodal model.

Building on this direction, Wang et al. [8] developed MinerU, an open-source dual-engine system combining Vision-Language Models (VLMs) and OCR to convert PDFs and other document types into structured Markdown and JSON, supporting 109 languages and automatic detection of scanned versus digitally-born PDFs. More recent work by Cassel et al. [7] demonstrates that end-to-end decoder transformer models outperform pipeline-based methods on accuracy metrics while offering greater flexibility on diverse document types.

Complementing these document-level findings, Cheng et al. [16] demonstrate that LLMs can capture structured semantic representations via prompting alone, without task-specific fine-tuning, though limitations emerge when handling complex argument structures. This supports the theoretical basis for using a vision LLM to recover structured Markdown from

rendered page images, while also setting bounded expectations for content types such as dense mathematical formulas.

These developments motivate the central premise of this thesis: that a *hybrid* extraction strategy combining text-layer extraction with image-based LLM understanding can achieve accuracy and structural fidelity that neither modality achieves alone.

Comparison of Extraction Strategies

Table 2 provides a preliminary comparison of the three fixed extraction strategies evaluated in this thesis alongside the adaptive approach proposed here, based on characteristics reported in the related literature [10, 11].

Table 2: Preliminary comparison of PDF extraction strategies

Criterion	Text-only	Image-only	Hybrid	Adaptive
Works on scanned PDFs	No	Yes	Yes	Yes
Preserves reading order	Partial	Yes (model-dep.)	Yes	Yes
Handles complex tables	Poor	Moderate	Good	Good
Handles mathematical formulas	Poor	Good	Good	Good
Computational cost	Low	High	Medium-High	Low-Medium
Structural fidelity	Low	Moderate	High	High
Dependency on LLM	None	Full	Partial	Selective

The table highlights the core trade-off that motivates this thesis: text-only extraction is fast and requires no LLM, but fails on any page containing images, formulas, or complex tables; image-only extraction handles all content types but applies a costly vision LLM call to every page regardless of need. The hybrid strategy improves structural fidelity but does not resolve the inefficiency of applying a fixed strategy uniformly. The adaptive approach addresses this by routing each page to the most appropriate strategy based on its detected content type, achieving the structural fidelity of the hybrid strategy while avoiding unnecessary LLM calls on plain-text pages.

1.3 Research Objectives and Questions

Given the identified problem and motivation, this thesis pursues the following primary research objective:

To design, implement, and evaluate a multimodal, content-aware system that converts PDF documents into structured Markdown by selecting an extraction strategy per page based on detected content type, powered by LM Studio as a local LLM runtime to ensure full reproducibility without dependence on external cloud services, and to assess whether this adaptive approach outperforms fixed text-only, image-only, and hybrid strategies with respect to accuracy, structural fidelity, and computational efficiency.

The system achieves this by analysing each PDF page individually and classifying it by content type before conversion. Pages identified as plain text are converted immediately using PyMuPDF without any LLM involvement, keeping processing fast and deterministic. Pages that contain visual or structured content — images, diagrams, mathematical formulas, tables, source code, headings embedded in layouts, or hyperlinks — are rendered as images and sent to LM Studio, where a locally-hosted vision-capable LLM performs the conversion using a prompt tailored to the detected content type. This routing decision is made per page, so a single document can combine both pathways depending on what each page contains.

This objective is operationalized through the following research questions:

- RQ1.** *Adaptivity:* Does a content-aware system that selects an extraction strategy per page based on detected content type (plain text, image, table, source code, formula, etc.) achieve higher accuracy and structural fidelity than any single fixed strategy applied to the full document?
- RQ2.** *Strategy comparison:* How do text-only, image-only, and hybrid extraction strategies individually compare in terms of content and structural accuracy across different PDF document categories?
- RQ3.** *Efficiency:* What are the computational costs (processing time, token usage, and number of LLM calls per document) of each strategy and of the adaptive system, and how do these trade off against accuracy gains?
- RQ4.** *Output quality:* How can the quality of generated Markdown be reliably assessed beyond standard n-gram metrics (BLEU, ROUGE), given that structured output requires evaluating heading hierarchy, list fidelity, and table reconstruction?

RQ5. Generalizability: Do the observed performance differences between strategies vary systematically across document types (scientific papers, forms, presentations, scanned documents)?

These questions are guided by the following hypotheses, which are evaluated against the experimental results in Chapter 6.

H1 — Adaptive superiority. The adaptive system will achieve higher overall structural fidelity and content accuracy than any single fixed strategy, because different page types have fundamentally different extraction requirements that no one strategy handles equally well.

H2 — Strategy-content fit. Text-only extraction will perform best on plain-text pages but degrade on image-heavy or formula-rich pages. Image-only extraction will show the opposite profile: it renders every page as an image and processes it through the vision LLM regardless of content type, meaning plain-text pages are penalised by unnecessary visual rendering and LLM inference, which introduces noise compared to direct text extraction.

H3 — Computational efficiency of the adaptive approach. Despite classifying each page before conversion, the adaptive system will have a lower average computational cost than the image-only or hybrid strategies, because plain-text pages are handled by PyMuPDF locally without any LLM call, and non-text pages are processed with prompts tailored to their content type rather than a single general-purpose prompt, reducing wasted inference on irrelevant instructions.

H4 — Hybrid as upper bound for mixed content. For documents with a balanced mix of text and visual content, the hybrid strategy will serve as a strong baseline that the adaptive system matches or exceeds by avoiding unnecessary LLM calls on purely textual pages.

1.4 Scope and Delimitations

This thesis focuses on PDF documents as the exclusive input format. The developed system is evaluated on English-language documents across four representative categories: academic papers, administrative forms, presentation slides, and scanned archival documents. While the system architecture is designed to be compatible with any LLM backend supporting a vision API, all experiments are conducted locally using LM Studio to ensure reproducibility without dependence on external cloud services.

The following aspects are explicitly out of scope:

- Real-time or streaming document processing.
- Support for languages other than English.
- Conversion of formats other than PDF (e.g., DOCX, HTML, PPTX).
- Fine-tuning of LLM models for the conversion task.
- Enterprise-scale deployment or cloud integration.

The evaluation focuses on Markdown as a single target format. Conversion to other structured formats (LaTeX, HTML, JSON) is not addressed, though the architecture supports such extensions through prompt modification.

1.5 Thesis Contributions

This thesis makes the following contributions to the field of document processing and multimodal AI systems:

1. **Content-aware adaptive conversion system.** A multimodal system that classifies each PDF page by content type — plain text, image, table, hyperlink, page number, source code, or mathematical formula — and applies the most appropriate extraction strategy per page, powered by LM Studio as a local LLM backend.
2. **Systematic comparison framework.** A structured experimental framework for comparing the adaptive system against three fixed strategies (text-only, image-only, hybrid) across four document categories, yielding actionable guidance on strategy selection based on document characteristics.
3. **Evaluation methodology for structured output.** A proposed evaluation methodology that extends beyond BLEU/ROUGE metrics to assess structural accuracy (heading hierarchy, list fidelity, table reconstruction) and semantic completeness.
4. **Empirical findings.** Empirical results demonstrating the trade-offs between accuracy, computational cost, and strategy choice across document categories, contributing to the growing body of work on LLM-based document understanding.
5. **Reproducible implementation.** A fully documented, reproducible implementation using open-source tools (PyMuPDF, LM Studio) with all prompt templates, evaluation scripts, and sample documents provided in the appendices.

1.6 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2 — Background and Fundamentals provides the theoretical foundation for this work, covering the PDF internal structure and parsing challenges, Markdown syntax and its use as a structured representation, the architecture and vision capabilities of modern LLMs, and a taxonomy of multimodal processing strategies.

Chapter 3 — Related Work surveys existing tools and research in three areas: traditional PDF processing tools (PyMuPDF, pdfplumber), modern conversion systems (Marker, LlamaParse, Nougat, MinerU), and LLM-based document understanding approaches. It identifies the limitations of current solutions that this thesis addresses.

Chapter 4 — Methodology describes the research design, dataset construction, ground truth creation process, and evaluation metrics. It introduces the comparative study framework and defines the experimental conditions under which the adaptive system and the three fixed strategies are evaluated.

Chapter 5 — System Design and Implementation presents the architecture of the developed system, including the per-page content classification module, PDF processing pipeline, LM Studio integration, prompt design per content type, and post-processing pipeline. It documents the technology stack and implementation decisions.

Chapter 6 — Results and Evaluation reports the experimental outcomes, including quantitative accuracy results by strategy and document type, qualitative case studies of success and failure modes, and a strategy comparison summary with recommended use cases.

Chapter 7 — Discussion interprets the results in light of the research questions, addresses threats to validity, and situates the findings within the broader related work.

Chapter 8 — Conclusion summarizes the findings, discusses limitations, and outlines directions for future work including support for additional document types, multilingual processing, and integration with newer LLM models.

2 Background and Fundamentals

2.1 PDF Format and Parsing Challenges

- PDF Internal Structure

- **PostScript heritage:** PDF evolved from PostScript, a page description language designed for rendering, not for storing structured information — this origin explains why PDF encodes content as drawing instructions rather than semantic elements [4]
- PDFs as “drawing instructions”: text stored as glyphs at (x, y) coordinates with no native concept of paragraph, heading, or list — all semantic structure must be reverse-engineered from geometric proximity, font size, and alignment cues [11]
- **PDF coordinate system:** the page origin is at the bottom-left corner with Y increasing upward, opposite to screen coordinates — positional sorting requires coordinate transformation and still fails on multi-column layouts [4]
- **Tagged vs. untagged PDFs:** ISO 32000-1 version 1.4 introduced “tagged PDF,” adding a logical structure tree that maps rendering commands to semantic roles such as headings and paragraphs and specifies reading order [2]; the vast majority of PDFs are untagged, meaning structure must be inferred purely from geometry, and even when tags exist, correct reading order still requires manual verification [2]
- Distinction between **native PDFs** (contain actual text objects with bounding boxes, amenable to structural parsing) and **scanned PDFs** (image-only pages requiring OCR and visual region detection before any text can be extracted) [11]
- Text Extraction Challenges (embedded fonts, encoding, layouts)
 - **Font encoding complexity:** PDF can embed fonts with custom encoding vectors, meaning character codes in the file do not directly map to Unicode — a root cause of garbled or incomplete text extraction, especially in older documents [4]
 - Flattened text extraction: loses paragraph boundaries, list structure, and semantic grouping [10]
 - Over-applying OCR to native PDFs: introduces noise and discards existing formatting metadata [10]
 - Multi-column layouts, footnotes, and mixed-language documents violate the assumption that positional order equals reading order
- Visual/Image-based Extraction Challenges
 - Images extracted without positional context lose their semantic relationship to surrounding text [11]

- Tables: PDF encodes no explicit row/column structure; complex layouts may require HTML fallback since Markdown lacks `rowspan`/`colspan` support [11]
- Code blocks: identifiable through monospaced font detection and consistent horizontal spacing, but easily fragmented by standard text extractors [10]

2.2 Markdown Format

- Introduction to Markdown syntax
- Markdown as a Structured Representation

2.3 Large Language Models

- Basic concepts and architecture
- Vision capabilities of modern LLMs
- Prompt Engineering for Structured Output

2.4 Multimodal Processing Strategies

- Text-only extraction
- Image-based extraction (visual parsing)
- Hybrid approaches combining both modalities

3 Related Work

3.1 Traditional PDF Processing Tools

- Text-based Parsers (PyMuPDF, pdfplumber, pypdfium2)
- OCR Solutions

3.2 Modern Document Conversion Tools

- Llama Parse, Marker, and Similar Tools
- Block-based conversion tools (e.g., pdftomarkdown.pro): treat pages as layout blocks with bounding boxes, sort by position, merge spans into paragraphs, and route scanned pages to OCR automatically [11]

- Limitations of Current Solutions: Darvishy [2] notes that no existing tool provides mechanisms to interpret and describe mathematical formulas or scientific graphs in PDFs — a gap that directly motivates the image-based strategy in the proposed system

3.3 LLM-based Document Understanding

- Vision-Language Models in Document Processing
- Multimodal Approaches for Layout Analysis

3.4 Evaluation Methods for Document Conversion

4 Methodology

4.1 Research Design

- Comparative Study Framework
- Research Questions Mapping

4.2 Dataset and Ground Truth Preparation

- Document Selection Criteria
- Document Categories and Characteristics
- Ground Truth Creation Process

4.3 Evaluation Metrics

- Structural Accuracy (heading hierarchy, lists, tables)
- Content Accuracy (text fidelity, completeness)
- Formatting Accuracy (links, code blocks, special elements)
- Limitations of N-gram Metrics (BLEU/ROUGE) for Markdown

4.4 Experimental Design

- Strategy Comparison (Text vs Image vs Hybrid)
- Model Variations and Parameters

- Baseline Comparisons

5 System Design and Implementation

5.1 Architecture Overview

- System components and workflow
- Integration with LLM backend (LM Studio)
- Data flow diagram

5.2 PDF Processing Module

- Text Extraction Pipeline
 - Native vs. scanned PDF detection and routing
 - Block-based extraction: sorting blocks by page and position, merging spans into paragraphs
- Image Rendering Pipeline
- Preprocessing and Normalization
 - Nested list reconstruction via bullet pattern detection and indentation comparison
 - Table handling: Markdown-compatible reconstruction where possible; HTML fallback for complex layouts with merged cells
 - Code block detection via monospaced font identification

5.3 LLM Integration

- Prompt Design for Each Strategy
- Message Formatting (Unimodal vs Multimodal)
- Configuration Parameters

5.4 Post-processing Pipeline

- Markdown cleanup and validation
- Element Extraction (Links, References)

- Quality Assurance Checks

5.5 Implementation Details

- Technology Stack
- LM Studio Integration
- Reproducibility Considerations

6 Results and Evaluation

6.1 Experimental Results Overview

6.2 Quantitative Analysis

- Overall Performance by Strategy
- Performance by Document Type
- Statistical Significance Testing

6.3 Qualitative Analysis

- Success Cases Analysis
- Failure Modes and Error Analysis
- Case Studies of Complex Documents

6.4 Strategy Comparison Summary

- Accuracy Trade-offs
- Computational Efficiency
- Recommended Use Cases

6.5 Limitations and Observations

7 Discussion

7.1 Interpretation of Results

- Why Strategies Succeed or Fail
- Impact of Document Characteristics

7.2 Threats to Validity

- Internal Validity
- External Validity

7.3 Practical Implications

7.4 Comparison with Related Work

8 Conclusion

8.1 Summary of Findings

8.2 Contributions and Impact

8.3 Limitations of the Study

8.4 Future Work

- Improvements to extraction strategies
- Support for additional document types
- Integration with other tools and pipelines
- Exploration of newer LLM models

References

- [1] Zhu, W., Sokhandan, N., Yang, G., Martin, S., & Sathyanarayana, S. (2022). DocBed: A multi-stage OCR solution for documents with complex layouts. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)*, pp. 12643–12649. AAAI Press.
- [2] Darvishy, A. (2018). PDF accessibility: Tools and challenges. In *Proceedings of the International Conference on Computers Helping People with Special Needs (IC-CHP 2018)*, LNCS vol. 10896, pp. 113–116. Springer. https://doi.org/10.1007/978-3-319-94277-3_20
- [3] Gao, L., Tang, Z., Lin, X., Liu, Y., Qiu, R., & Wang, Y. (2011). Structure extraction from PDF-based book documents. In *Proceedings of the 11th Annual International ACM/IEEE Joint Conference on Digital Libraries (JCDL 2011)*, pp. 11–20. ACM. <https://dl.acm.org/doi/10.1145/1998076.1998079>
- [4] Whittington, J. (2012). *PDF Explained: The ISO Standard for Document Exchange*. O’Reilly Media.
- [5] Subramani, N., Matton, A., Greaves, M., & Lam, A. (2020). A survey of deep learning approaches for OCR and document understanding. *arXiv preprint arXiv:2011.13534*. <https://arxiv.org/abs/2011.13534>
- [6] Blecher, L., Cucurull, G., Scialom, T., & Stojnic, R. (2023). Nougat: Neural Optical Understanding for Academic Documents. *arXiv preprint arXiv:2308.13418*. <https://arxiv.org/abs/2308.13418>
- [7] Cassel, T., et al. (2025). Accelerating end-to-end PDF to Markdown conversion through assisted generation. *arXiv preprint arXiv:2512.18122*. <https://arxiv.org/abs/2512.18122>
- [8] Wang, B., et al. (2024). MinerU: An open-source solution for precise document content extraction. *arXiv preprint arXiv:2409.18839*. <https://arxiv.org/abs/2409.18839>
- [9] Smith, R. (2007). An overview of the Tesseract OCR engine. In *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, vol. 2, pp. 629–633. IEEE. <https://doi.org/10.1109/ICDAR.2007.4376991>
- [10] Adhikari, N.S., & Agarwal, S. (2024). A comparative study of PDF parsing tools across diverse document categories. *arXiv preprint arXiv:2410.09871*. <https://arxiv.org/abs/2410.09871>

- [11] Zhang, Q., et al. (2024). Document parsing unveiled: Techniques, challenges, and prospects for structured information extraction. *arXiv preprint arXiv:2410.21169*. <https://arxiv.org/abs/2410.21169>
- [12] Zhang, B., et al. (2026). ParseBench: A document parsing benchmark for AI agents. *arXiv preprint arXiv:2604.08538*. <https://arxiv.org/abs/2604.08538>
- [13] Khan, A.A., et al. (2024). Developing retrieval augmented generation (RAG) based LLM systems from PDFs: An experience report. *arXiv preprint arXiv:2410.15944*. <https://arxiv.org/abs/2410.15944>
- [14] Saraiva, B.M., et al. (2025). Rxiv-Maker: An automated template engine for streamlined scientific publications. *arXiv preprint arXiv:2508.00836*. <https://arxiv.org/abs/2508.00836>
- [15] Cone, M. (2019). *The Markdown Guide*. Available at: <https://www.markdownguide.org>
- [16] Cheng, N., Yan, Z., Wang, Z., Li, Z., Yu, J., Zheng, Z., Tu, K., Xu, J., & Han, W. (2024). Potential and limitations of LLMs in capturing structured semantics: A case study on SRL. In *Proceedings of the 20th International Conference on Advanced Intelligent Computing Technology and Applications (ICIC 2024)*, LNAI vol. 14867, pp. 50–61. Springer. https://doi.org/10.1007/978-981-97-5663-6_5

A Implementation Code Snippets

B Complete Results Tables

C Sample Documents and Outputs

D Prompt Templates